

# BUCKET SORT

Equipe: Bruna Kaori, Daniel Durante, Igor Cantieri, Lucas Gabriel e Rafael Moreira.

Professor: Clayton Kossoski

# SUMÁRIO

- Introdução e conceito
- Abordagem Scatter-Gather
- Funcionamento do algoritmo
- Fluxo de execução
- Complexidade do algoritmo
- Pontos fortes e fracos
- Aplicações Práticas
- Código Fonte
- Referências

# INTRODUÇÃO E CONCEITO

Bucket Sort é um algoritmo de ordenação que divide os elementos de um array não ordenado em vários grupos chamados buckets.

# ABORDAGEM SCATTER-GATHER

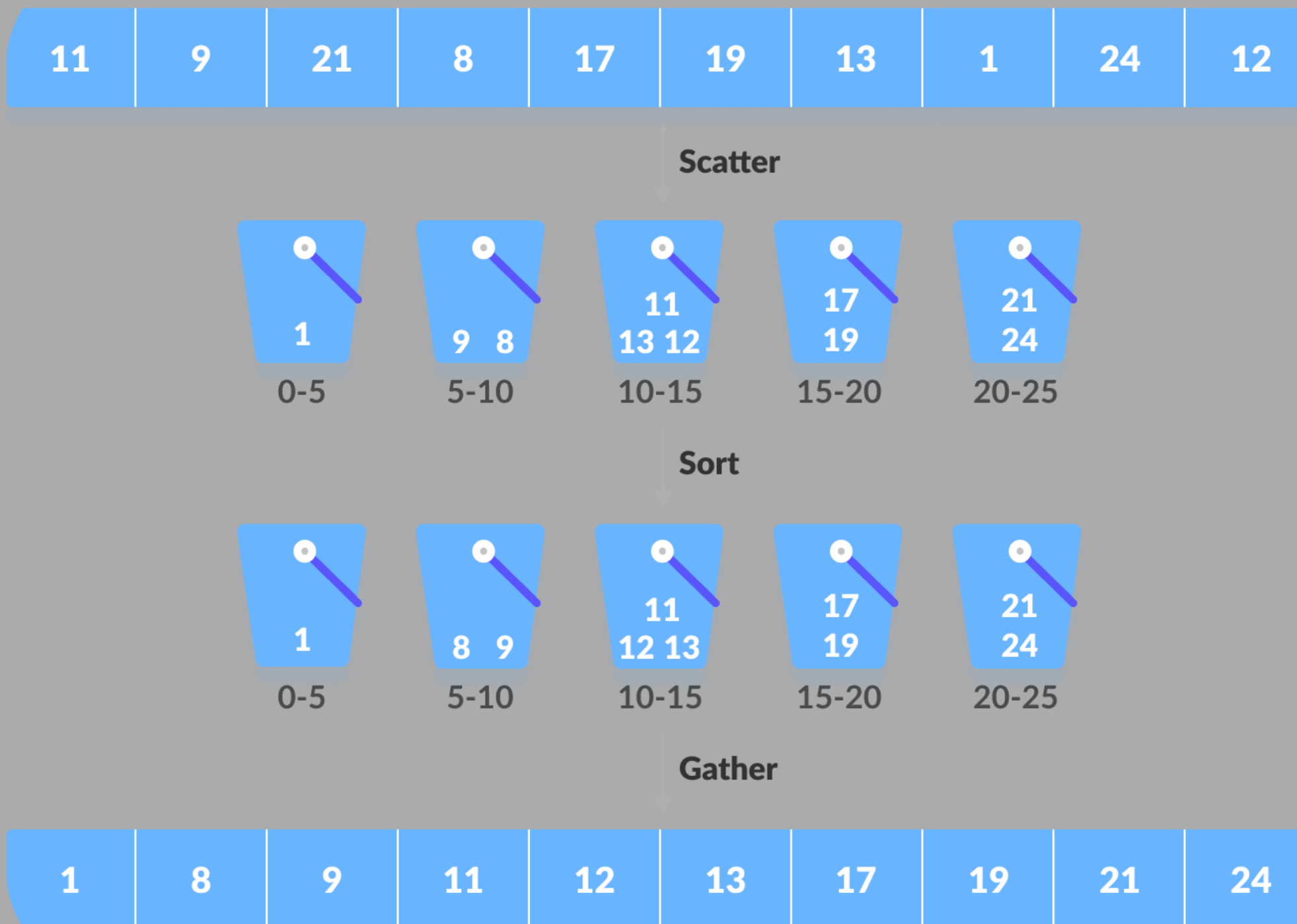
## 1. Espalhar(SCATTER)

O conjunto de dados inicial é particionado em subconjuntos chamados de baldes (buckets). A distribuição dos elementos é feita através de uma função de mapeamento (ou cálculo de índice) baseada no intervalo de valores.

## 2. Juntar(GATHER)

Cada balde é ordenado individualmente utilizando um algoritmo secundário (como Insertion Sort ou QuickSort). Por fim, os elementos de todos os baldes são concatenados em ordem sequencial para formar a lista final ordenada.

# FUNCIONAMENTO DO ALGORITMO



- **Divisão em Intervalos:** Define-se um número  $k$  de baldes cobrindo o alcance total dos dados.
- **Mapeamento Direto:** Cada valor passa por um cálculo simples de índice para determinar seu balde destino.
- **Ordenação Local:** Baldes individuais contêm poucos elementos, tornando a ordenação interna rápida.
- **União Linear:** A travessia dos baldes gera o resultado ordenado em tempo reduzido.

# FLUXO DE EXECUÇÃO

1

## Inicialização

Criação de N baldes vazios para armazenar as listas dinâmicas.

2

## Distribuição

Iteração sobre o array inserindo cada elemento no balde correto.

3

## Ordenação

Aplicação de ordenação individual em cada um dos baldes.

4

## Concatenação

Junção de todos os baldes para compor o array final.

# COMPLEXIDADE DO ALGORITMO

| Cenário     | Complexidade Temporal | Complexidade Espacial | Condição de Ocorrência                                   |
|-------------|-----------------------|-----------------------|----------------------------------------------------------|
| Melhor Caso | $O(n + k)$            | $O(n + k)$            | Elementos distribuídos uniformemente entre os baldes.    |
| Caso Médio  | $O(n)$                | $O(n + k)$            | Distribuição aleatória sem grandes concentrações locais. |
| Pior Caso   | $O(n^2)$              | $O(n + k)$            | Todos os elementos colocados no mesmo balde.             |

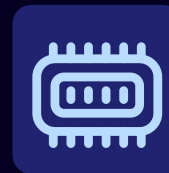
n: número de elementos  
k: número de baldes alocados

# PONTOS FORTES E FRACOS



## Desempenho

Alcança tempo linear de ordenação quando os dados de entrada seguem uma distribuição uniforme contínua.



## Overhead de Memória

Exige alocação adicional para os baldes, tornando o consumo de espaço superior a algoritmos in-place.



## Sensibilidade a Dados

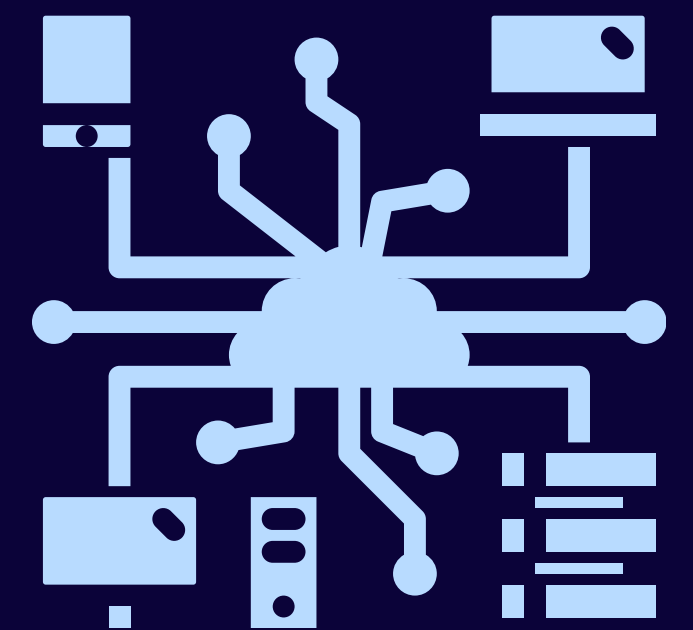
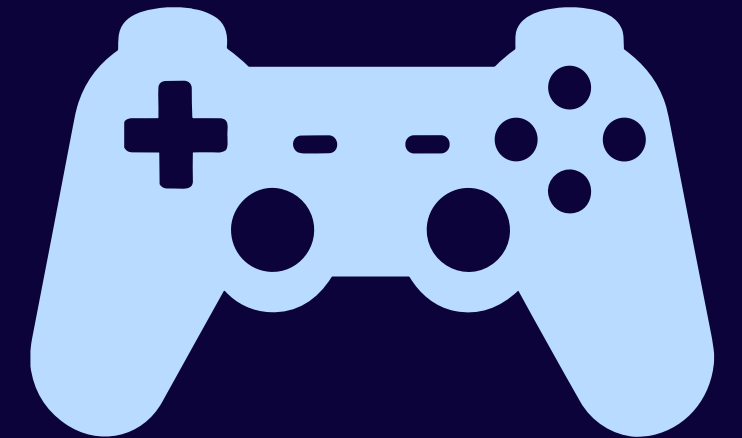
Se os dados forem muito agrupados (clustered), a eficiência decai rapidamente para o pior caso.

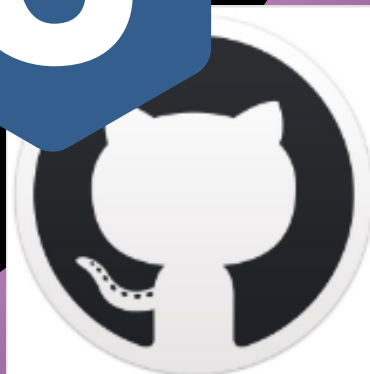
# APLICAÇÕES PRÁTICAS

O bucket sort é ideal para ordenar dados contínuos com intervalo conhecido e distribuição uniforme (ponto flutuante).

Exemplos:

- Processamento de Dados e Planilhas
- Sistemas de Gráficos e Jogos
- Simulações Computacionais





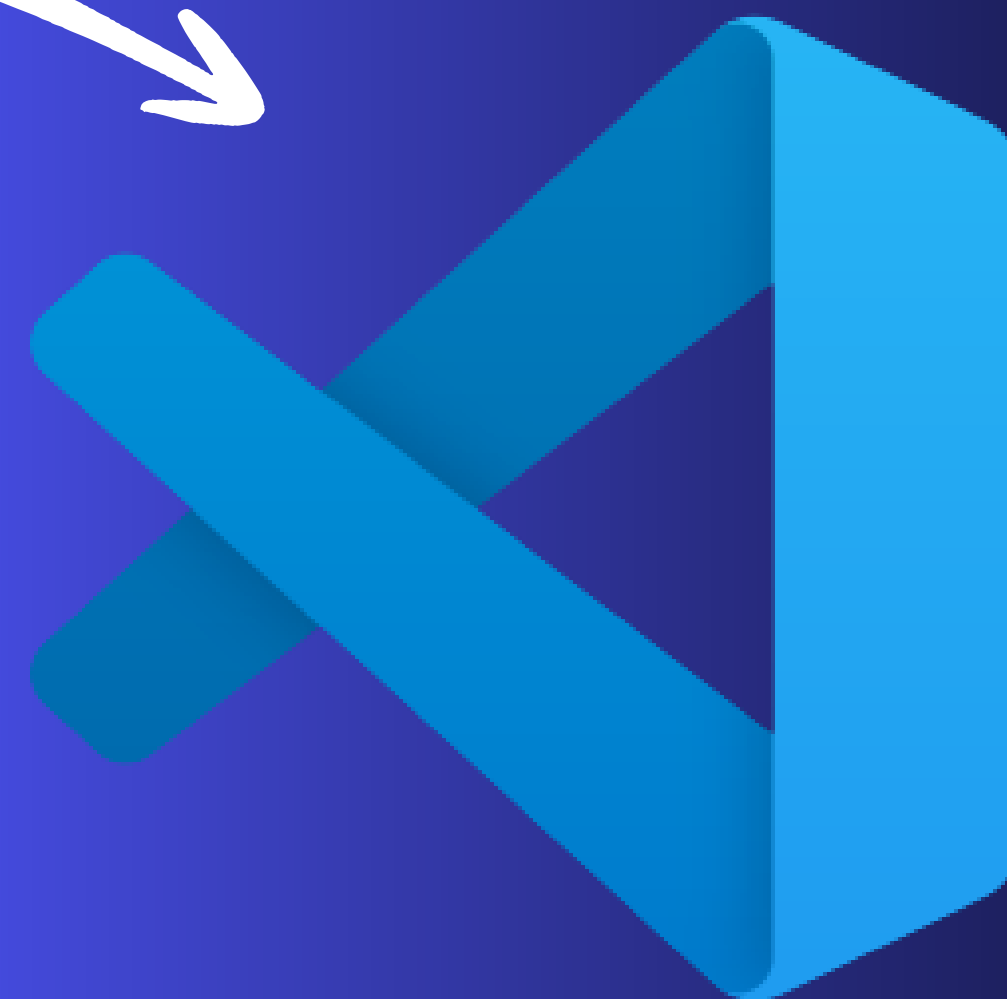
**tkaoribruna/bucket-sort**

Contribute to tkaoribruna/bucket-sort development by creating an account on GitHub.



GitHub

# CÓDIGO FONTE



# ARRAY INICIAL

7

2

9

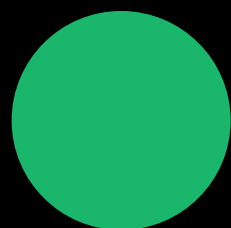
4

1

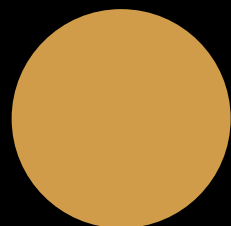
6

3

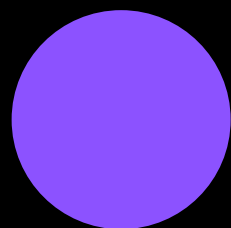
8



Bucket 0: 0 - 3

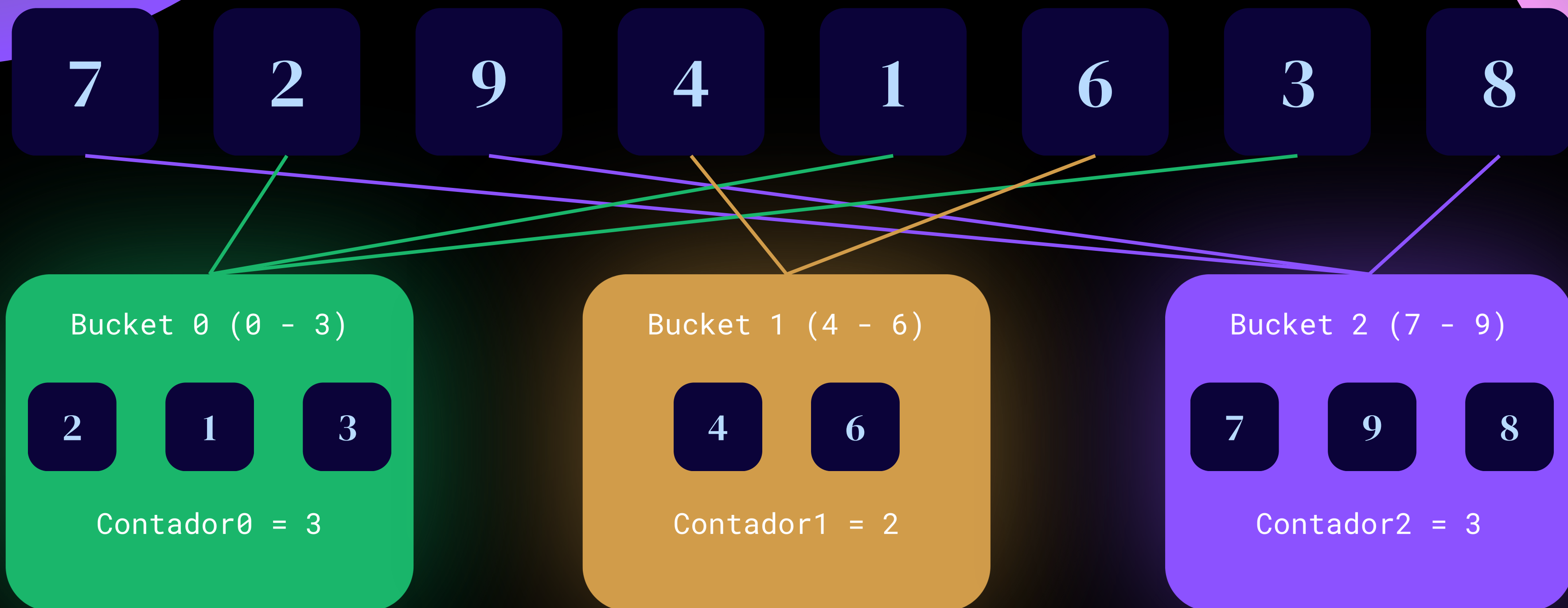


Bucket 1: 4 - 6



Bucket 2: 7 - 9

# DISTRIBUINDO OS ELEMENTOS



# ORDENANDO O VALOR INTERNO DE CADA BUCKET

Bucket 0 (0 - 3)

2

1

3

Bucket 1 (4 - 6)

4

6

Bucket 2 (7 - 9)

7

9

8

# JUNTANDO OS RESULTADOS

1

Primeiro o `bucket 0: 0 - 3`

2

Depois o `bucket 1: 4 - 6`

3

Por fim o `bucket 2: 7 - 9`

# ARRAY FINAL(ORDENADO)

Após a aplicação do algoritmo de ordenação bucket sort.

1

2

3

4

6

7

8

9

# REFERÊNCIAS

- PROGRAMIZ. Bucket Sort Algorithm. Parewa Labs, [s. d.]. Disponível em: <https://www.programiz.com/dsa/bucket-sort>. Acesso em: 8 set. 2026.
- PYTHON READ. Bucket Sort Algorithm. PythonRead, 2023. Disponível em: <https://pythonread.github.io/dsa/bucket-sort.html>. Acesso em: 8 set. 2026.

# OBRIGADO PELA ATENÇÃO!

Dúvidas?

Equipe: Bruna Kaori, Daniel Durante, Igor Cantieri, Lucas Gabriel e Rafael Moreira.

Professor: Clayton Kossoski